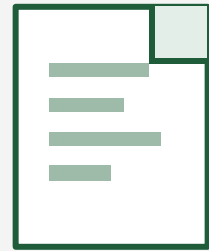

Software Architecture and Design

Oscar Chaparro

Too simple?

THE SUBMISSION



Challenge: a CLI tool that calls an API and pages through the data. One submission: ~30 lines of Python, Python, the simplest way to meet the requirements. ✓ **all requirements**

THE SECOND REVIEWER

Proposed as the reference for a 10/10. The other reviewer wouldn't pass it to an interview — not enough sophisticated language features. **“Too simple.”**

REJECTED

Why simple wins: every part is a way to fail



API down → 1 way to fail



API down **stale cache** **state out of sync** **retry storm**

→ many

“Great software design looks simple because it eliminates as many failure modes as possible during the design stage.”

S. Goedecke, “Great software design looks underwhelming,” Mar 2025 · seangoedecke.com/great-software-design



Agenda

1 What architecture and design are

2 Design principles

3 From quality attributes to design decisions

4 Architectural styles

5 Designing inside an existing codebase

6 Recording design decisions

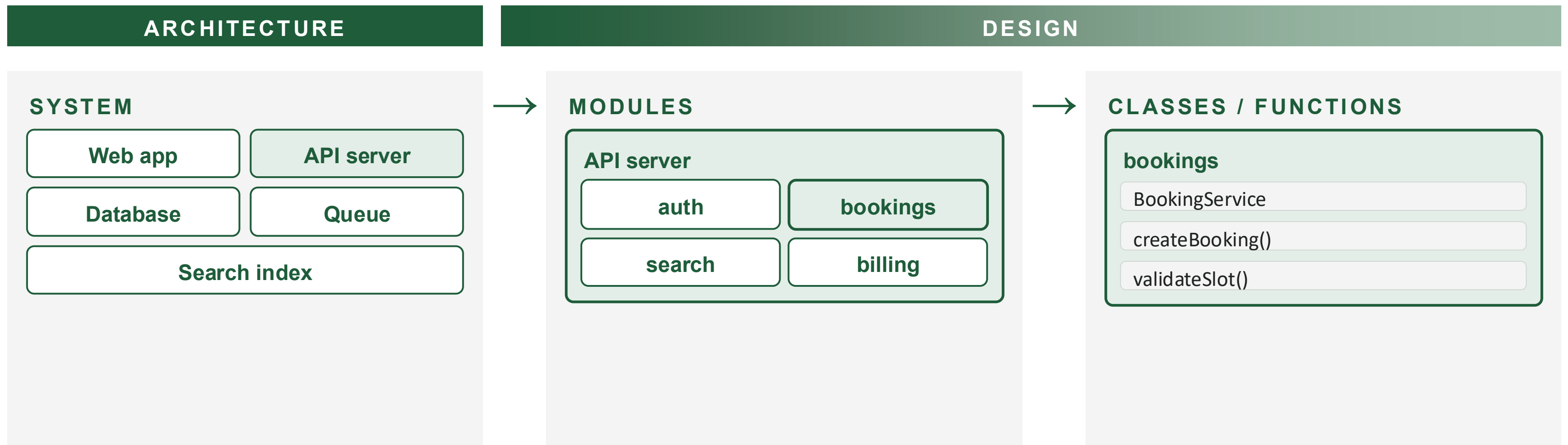
7 Agents and design



What architecture and design are

Structure, decisions, and reasons

Architecture and design



Architecture: the main parts of a system, how they connect, and the key decisions behind them — **and why**.

Design: the same questions at a smaller scale.

One system, several views

Module view

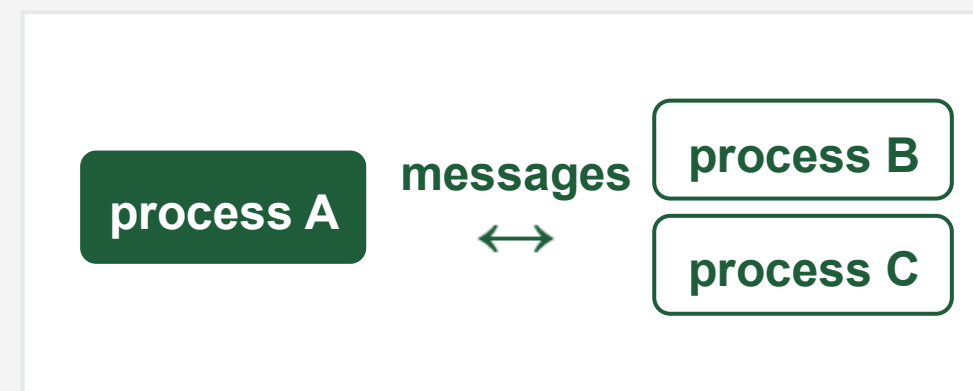


How is the code split, and what may depend on what? → **where do I make a change?**

Chromium layers

VS Code

Runtime view

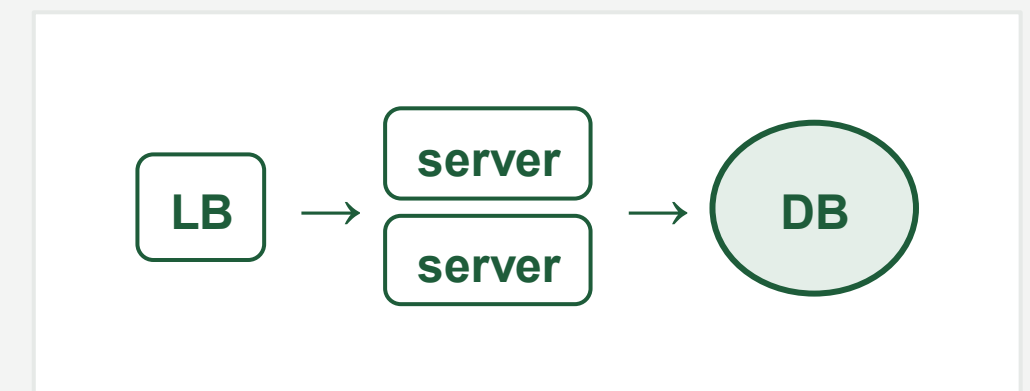


What runs, and how do the parts talk while it runs? → **what fails together?**

Chromium processes

VS Code extension host

Deployment view



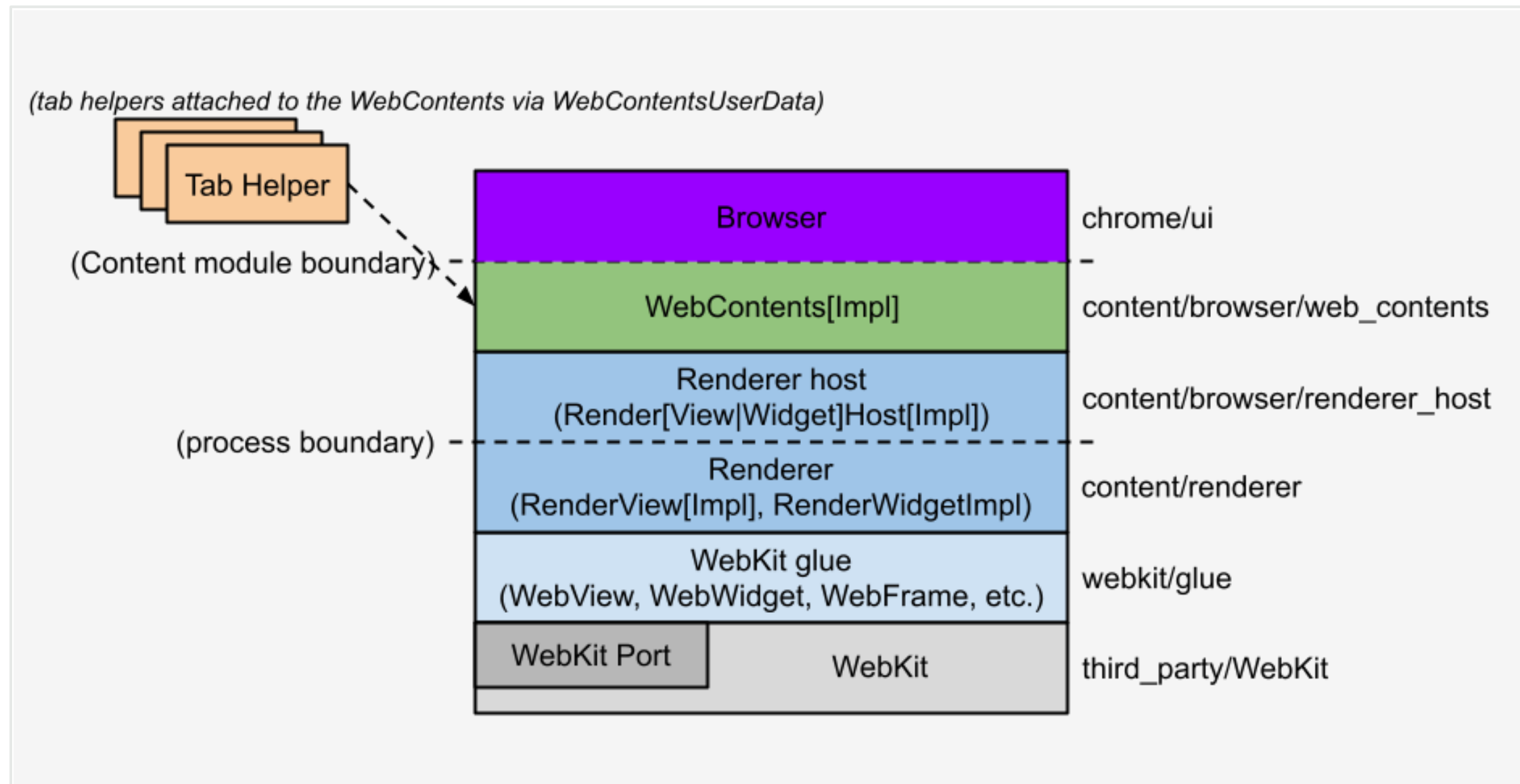
Which machines does it run on? → **where does it run, what does it cost?**

Stack Overflow

Different questions → different views. All of them are architecture.

Chromium: module view

MODULE VIEW



Source: chromium.org — “How Chromium Displays Web Pages” · CC BY 2.5

KEY COMPONENTS

WebKit (+ Port)

rendering engine; the Port connects it to OS services (loading, graphics)

Glue

converts WebKit types to Chromium types — the “WebKit embedding layer”

Renderer / Render host

carry commands and notifications across the process boundary

WebContents

reusable component that renders HTML into a view

Browser

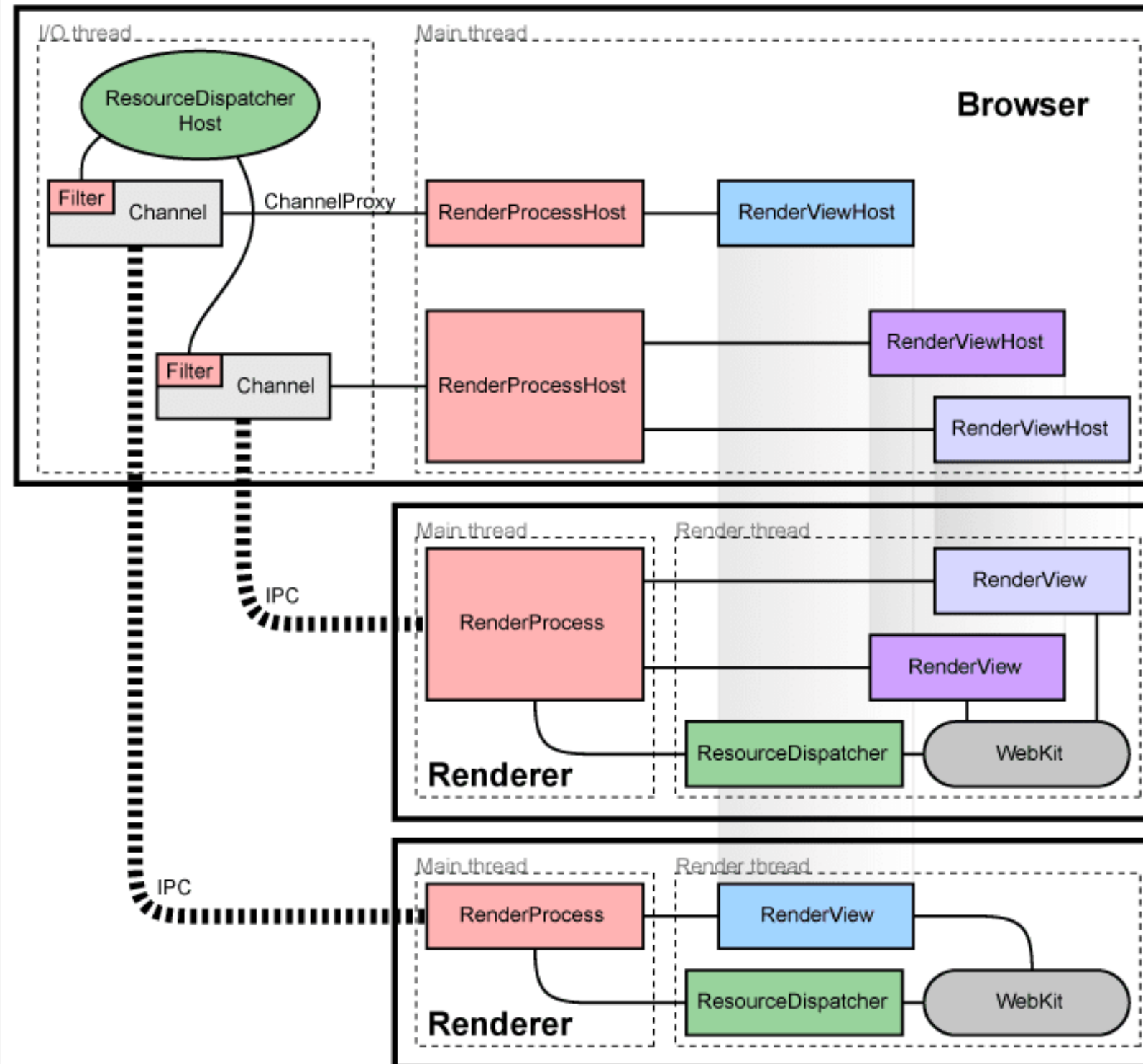
the browser window; holds many WebContents

Tab helpers

small add-ons attached to a WebContents (favicons, infobars)

Rule: no layer depends on a higher layer.

Chromium: runtime view



RUNTIME VIEW

KEY COMPONENTS

Browser process

one; runs the UI

Renderer process

one per site; runs WebKit in a sandbox

RenderProcessHost

browser-side handle on a renderer

Channel + IPC

async messages between processes

ResourceDispatcher (Host)

network requests go through the browser

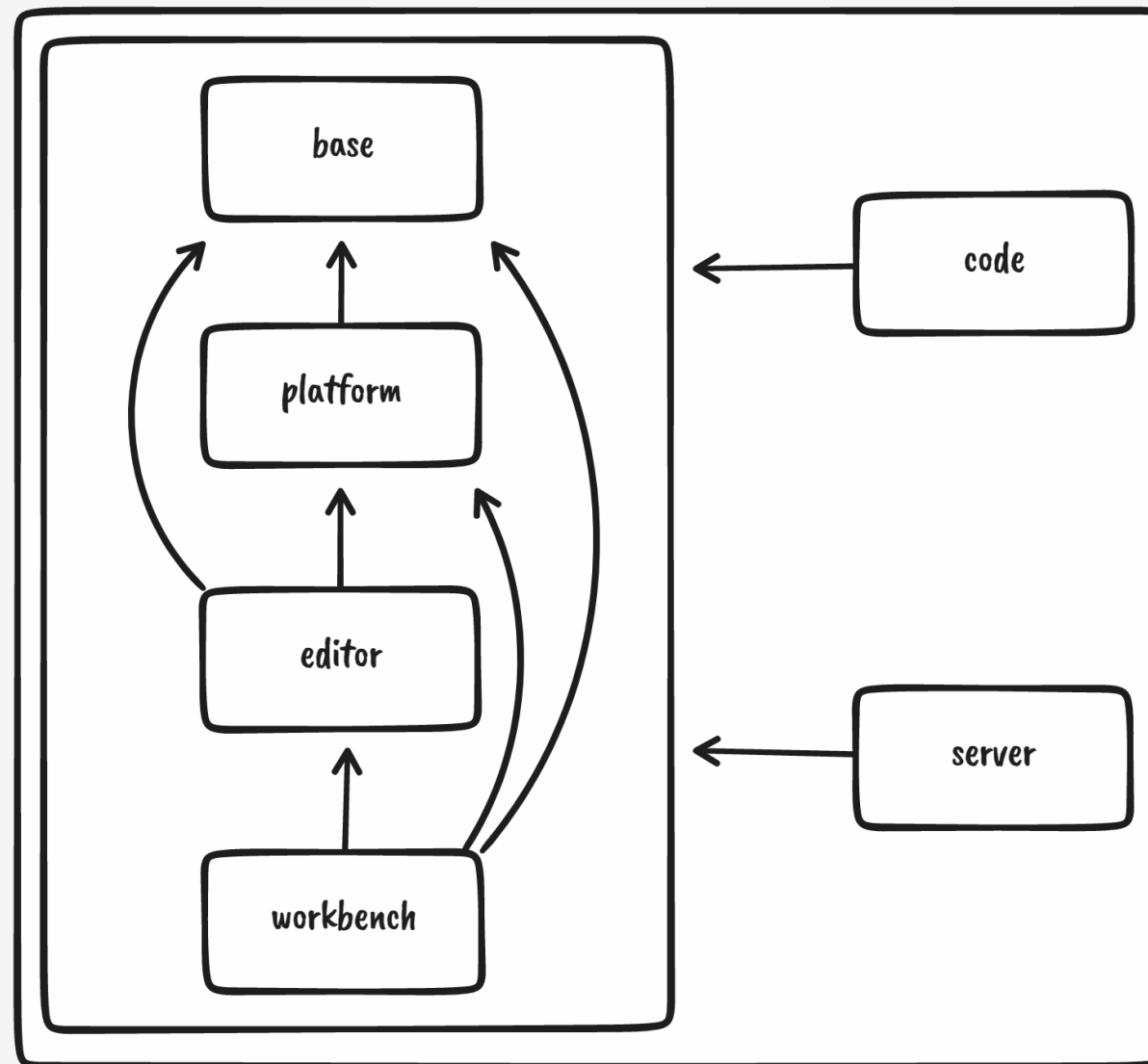
Why: a crash shows a “sad tab”, not a dead browser. **Cost:** more memory.

IPC = inter-process communication:
messages between processes, which don't share memory.

Source: chromium.org — “Multi-process Architecture” design document · CC BY 2.5

VS Code: module view

MODULE VIEW



KEY COMPONENTS

base

general utilities and UI building blocks

platform

service injection and shared services

editor

the Monaco editor core

workbench

the IDE UI: editors, panels, status bar

code / server

starts the app: Electron main process, windows, CLI / remote server

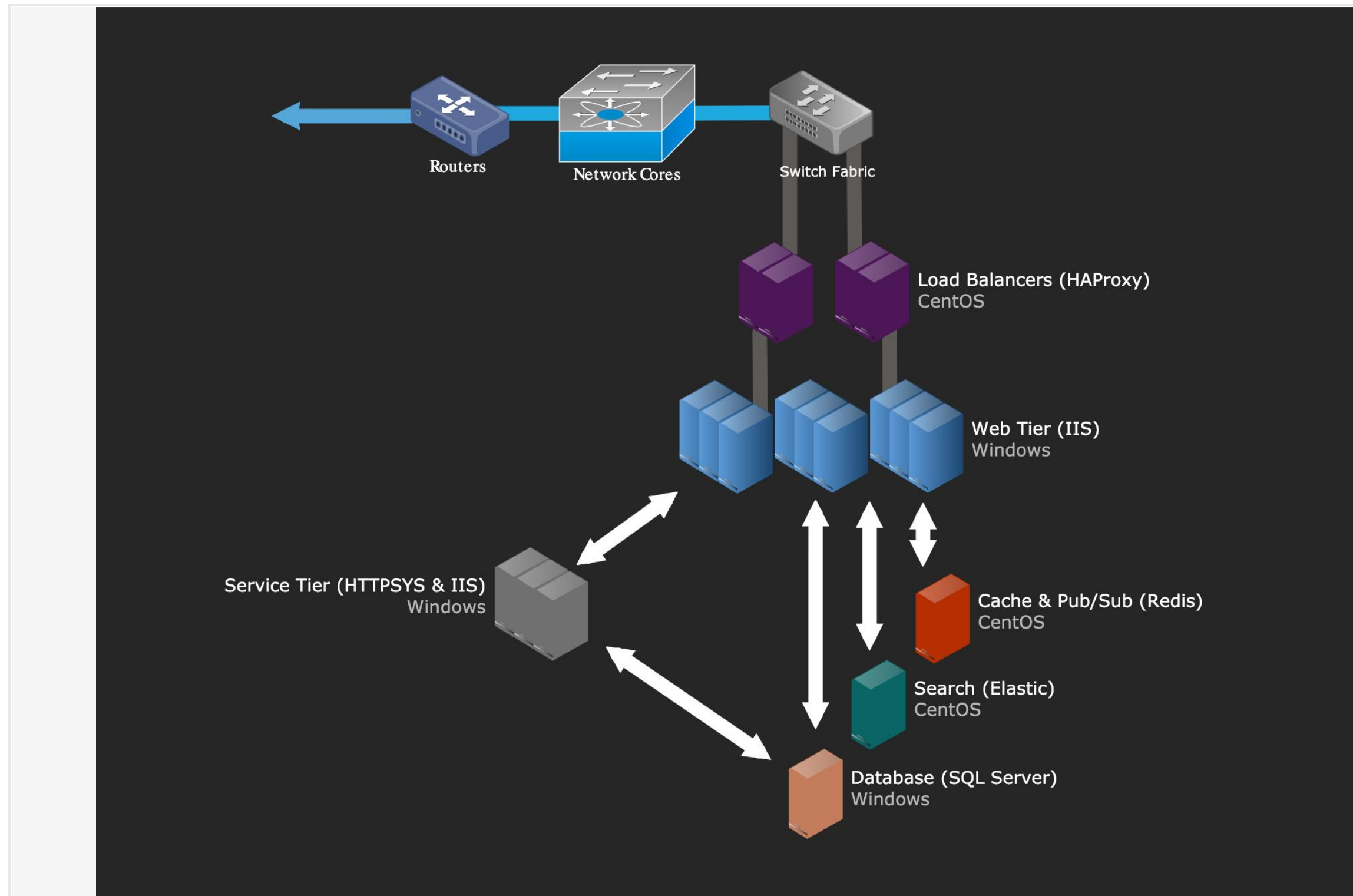
extension host

separate process; extensions use only the extension API

Source: github.com/microsoft/vscode/wiki — “Source Code Organization”

Rule: a layer uses only the layers below it. Extensions never touch the core directly.

Stack Overflow: deployment view



DEPLOYMENT VIEW

KEY COMPONENTS

Load balancers (HAProxy)

send each request to a web server

Web tier (IIS, ASP.NET MVC)

the main application that serves the sites

Service tier

separate servers for the tag engine and internal APIs — heavy work kept off the web tier

SQL Server

the main data store

Redis

shared cache and pub/sub between servers

Elasticsearch

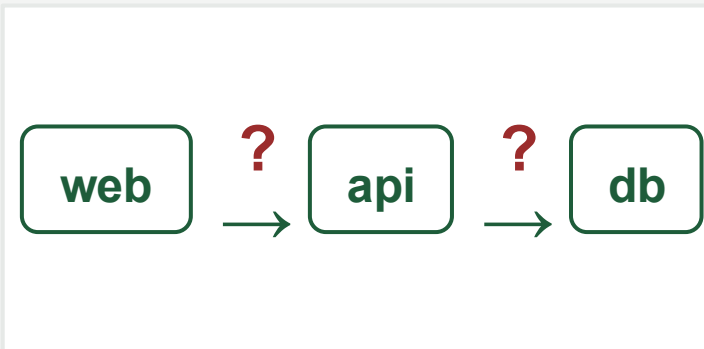
search

A huge site, mostly one application,
on few servers — 2016 snapshot.

Source: N. Craver, "Stack Overflow: The Architecture – 2016 Edition," nickcraver.com

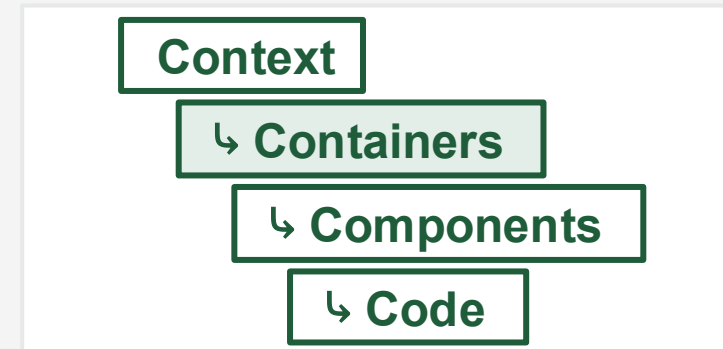
How people draw architectures

Boxes and arrows



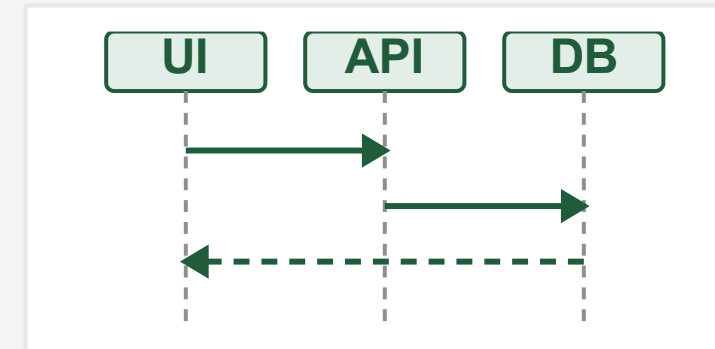
most common; fast, but what does an arrow mean?

C4



four zoom levels: context → containers → components → code

UML



mostly sequence, component and deployment diagrams

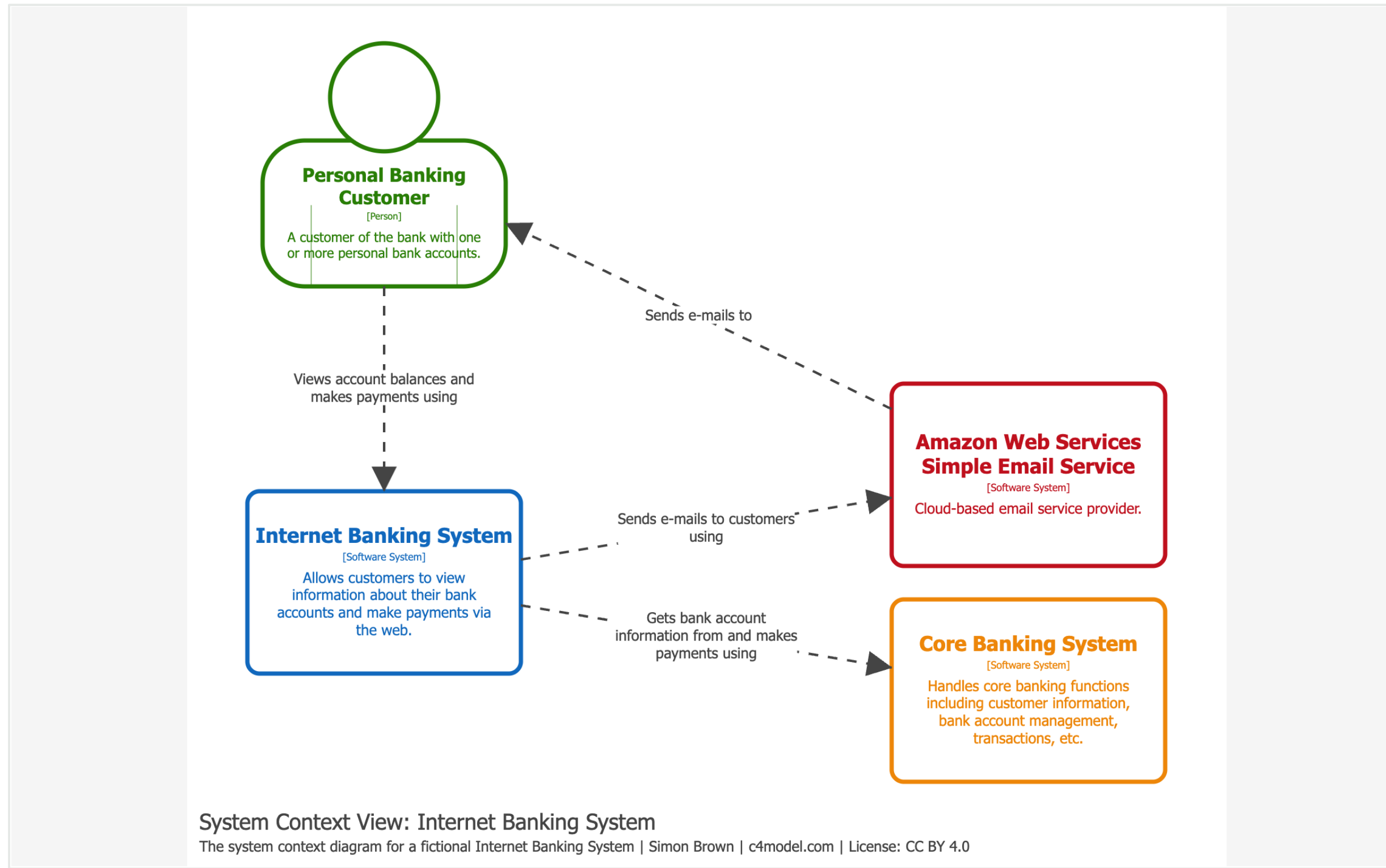
Diagrams as code

```
```mermaid
graph LR
 web --> api
 api --> db
```
```

Mermaid, PlantUML, Structurizr — kept in the repo

Whatever you use, add a legend: say what a box and an arrow mean.

C4: system context diagram



LEVEL 1 · CONTEXT

KEY COMPONENTS

Person

a user or role (the customer)

Software system (in scope)

the system we are building: Internet Banking

Existing systems

outside systems it talks to: mainframe, e-mail

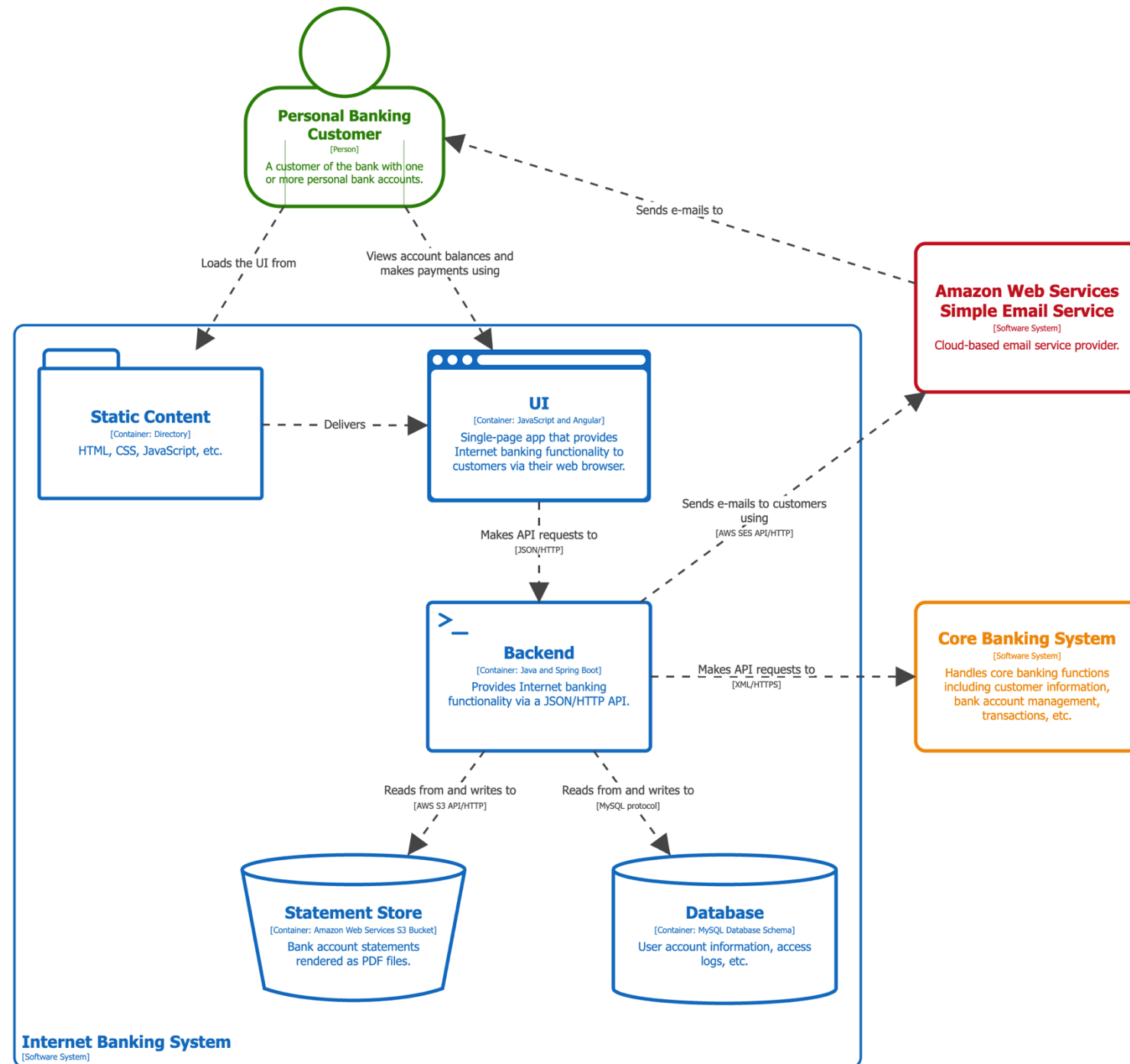
Arrows

labelled with what happens

**Big picture: no technology yet.
Readable by non-technical people.**

Source: Simon Brown, c4model.com · CC BY 4.0

C4: container diagram



Container View: Internet Banking System
The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

LEVEL 2 · CONTAINERS

KEY COMPONENTS

Container

an application or data store that runs on its own

Web app, single-page app, mobile app

what customers use

API application

server-side logic

Database

stores the data

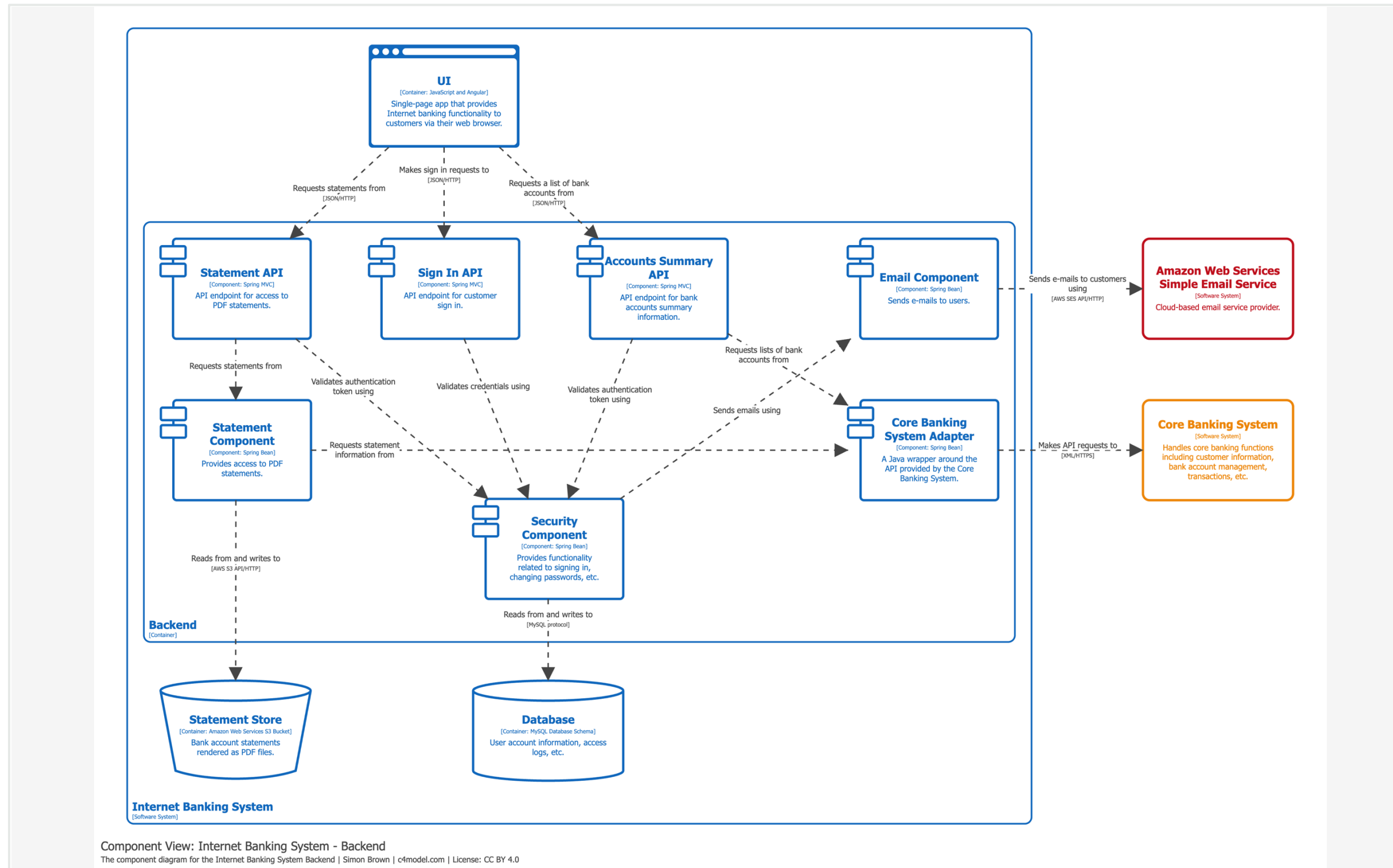
Labels

each box names its technology; each arrow its protocol

Zoom in: the main parts, their technology, and how they talk.

Source: Simon Brown, c4model.com · CC BY 4.0

C4: component diagram



LEVEL 3 · COMPONENTS

KEY COMPONENTS

Container in scope

the API application, drawn as a boundary

Components

its main parts: controllers, security, facades

Responsibilities

each box says what it does and its technology

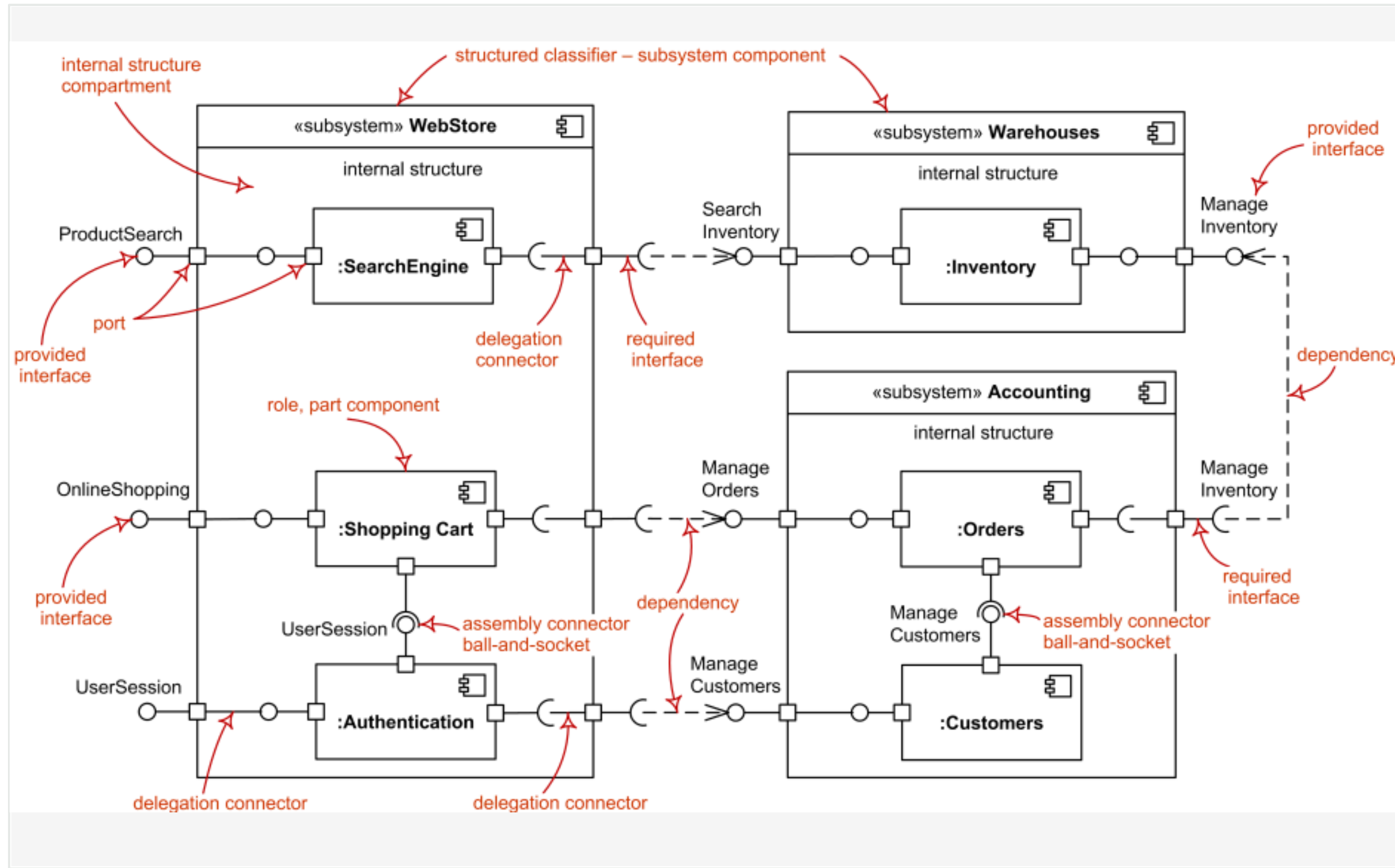
Arrows

calls between components and to containers

Zoom in again: inside one container.
Optional.

Source: Simon Brown, c4model.com · CC BY 4.0

UML: component diagram



UML 2

KEY COMPONENTS

Component

box with the «component» keyword or icon

Provided interface (ball)

a service the component offers

Required interface (socket)

a service the component needs

Port

a point where interfaces attach

Assembly connector

ball meets socket: one part uses another

Delegation connector

forwards from a port to an inner part

Standard notation: components and the interfaces between them.

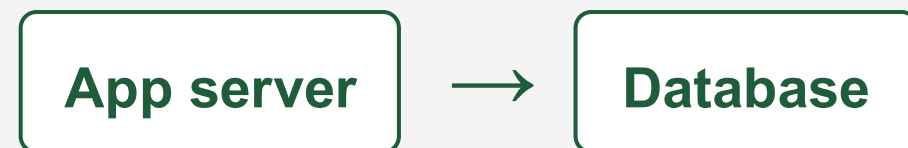
Source: K. Fakhroutdinov, uml-diagrams.org — “UML Component Diagrams Reference”

Quality attributes drive the structure

Feature: “Find documents by a word.”

TWO ARCHITECTURES

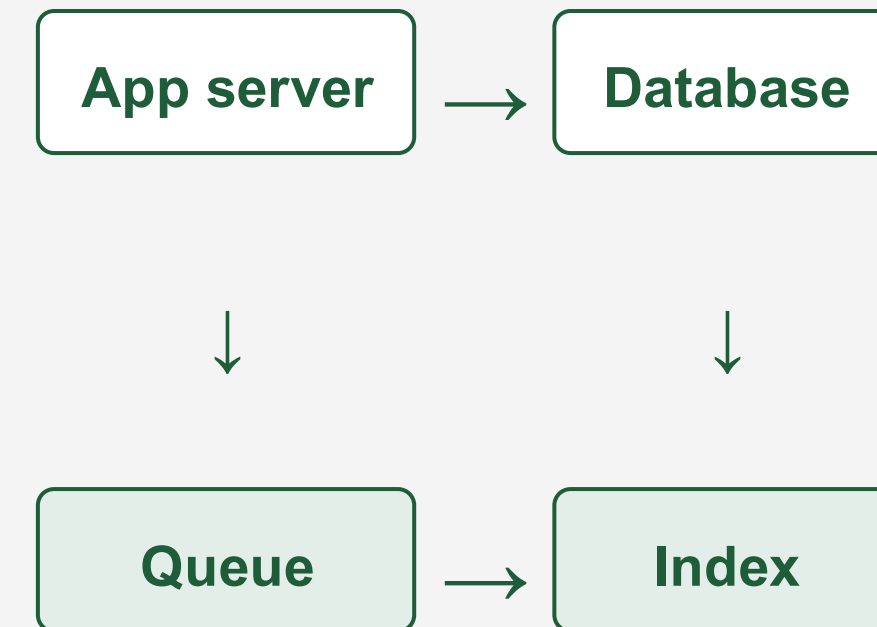
A few hundred documents



→ one query on the database

scale: small

Millions of documents, results in < 500 ms



→ full-text index, background indexing

performance: < 500 ms at millions of docs

Two laws

1 TRADE-OFFS

Everything in software architecture is a trade-off.

Cache: faster reads ↔ data may be stale

Separate processes: a crash stays contained ↔ more memory

Search index: fast search ↔ results lag behind updates

2 WHY?

Why is more important than how.

The code shows how, not why.

You can see the queue, but not that it keeps search under 500 ms.

Without the why, nobody can safely change it.

Someone removes the “unnecessary” queue and search slows down.

The why is what you re-check when things change.

If the reason no longer holds, the design can change.

— Richards & Ford, *Fundamentals of Software Architecture*

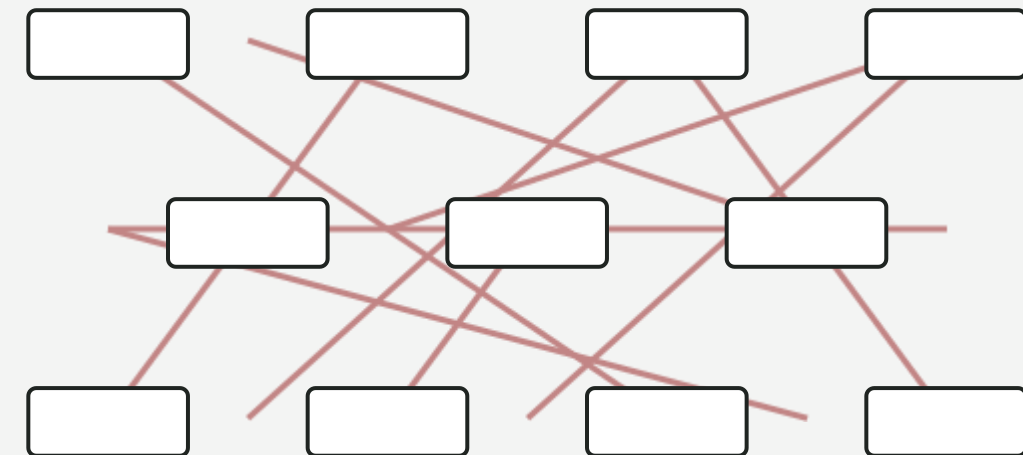


Good design goes unnoticed



✓ 0 incidents

Good design: nothing happens. Changes are easy. Looks obvious afterwards.



X pager: incident #14

Bad design: impressive, many parts, clever tricks — and frequent incidents.

“Good design is self-effacing: bad design is often more impressive than good.”

— S. Goedecke, 2025